Project - High Level Design

on

Construction Planning Assistant Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Ansh Kumbhare	EN22CS301140
02	Anshul Sen	EN22CS301158
03	Anshul Yadav	EN22CS301159
04	Armi Bhawsar	EN22CS301194
05	Atharv Dubey	EN22CS301229

Group Name: 06D2

Project Number: AAI-18 **Industry**

Mentor Name: Mr. Suraj Naik

University Mentor Name: Prof. Ajeet Singh Rajput

Academic Year: 2025-2026

1.Introduction:-

The Construction Planning Assistant Agent is an Agentic AI-based system designed to automate and optimize construction project planning. Unlike traditional static report generation systems, this solution uses multi-agent reasoning and decision-making to decompose complex construction goals into structured execution plans.

The system accepts a high-level goal (e.g., "Permit Requirements", "Project Timeline Planning") and autonomously:

- Breaks it into smaller actionable tasks
- Validates resources using tool APIs (mock)
- Generates optimized execution schedules

This enhances decision-making, planning accuracy, and automation in construction workflows.

1.1 Scope of the Document

This document covers the design and implementation of the Construction Planning Assistant Agent, including:

- Agent-based architecture using CrewAI / LangChain
- Multi-step reasoning loop (Think → Act → Observe → Repeat)
- Integration of Vector Database (FAISS) for knowledge retrieval
- Use of LLM APIs for intelligent planning
- Simulation of real-world tools (permit checker, cost estimator, etc.)
- Generation of structured planning outputs and schedules

1.2 Intended Audience

- Construction Project Managers
- Site Engineers & Planners
- AI/ML Developers working on Agentic Systems
- Stakeholders requiring automated planning solutions

1.3 System Overview

The system acts as an **intelligent planning assistant** that: •

Accepts a user goal

- Uses an **Orchestrator Agent** to understand intent
- Activates a **Planner Agent**
- Executes a reasoning loop with tool interactions

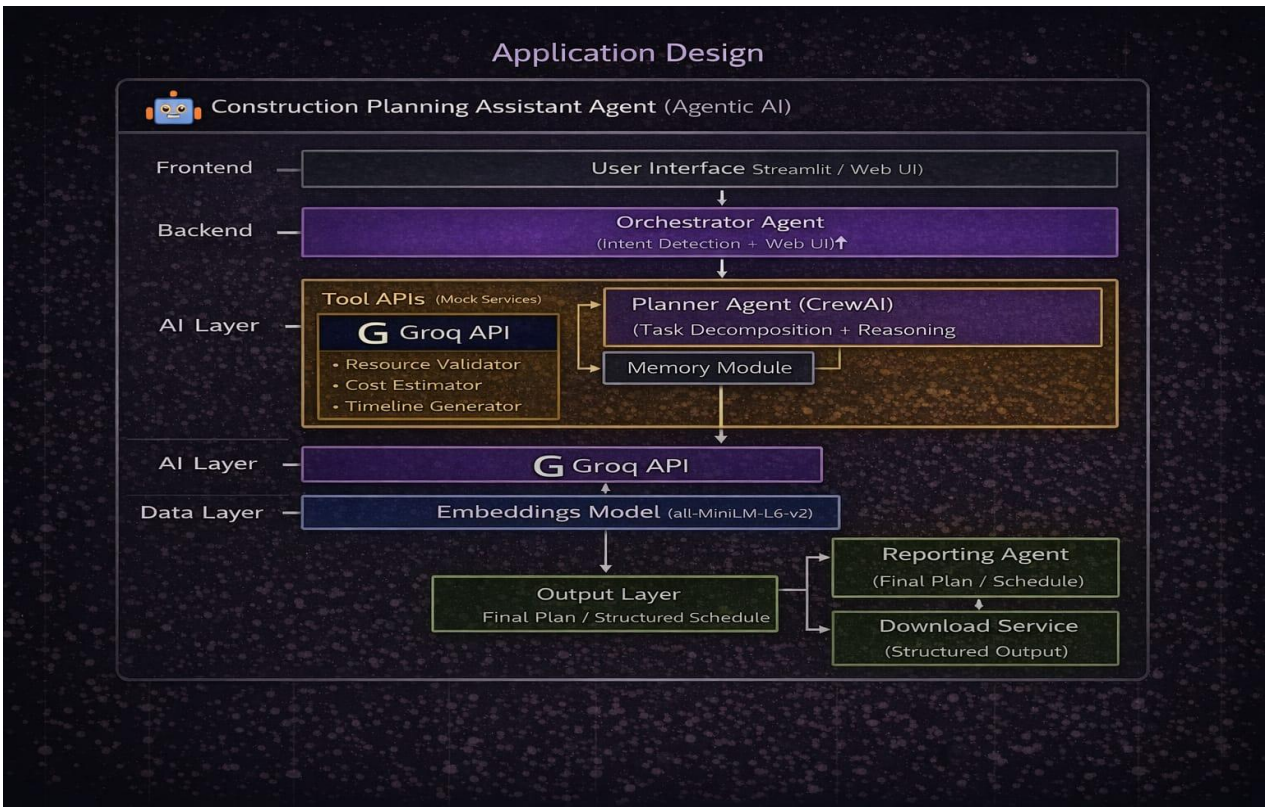
• Generates a **final construction plan and schedule**

2. System Workflow:

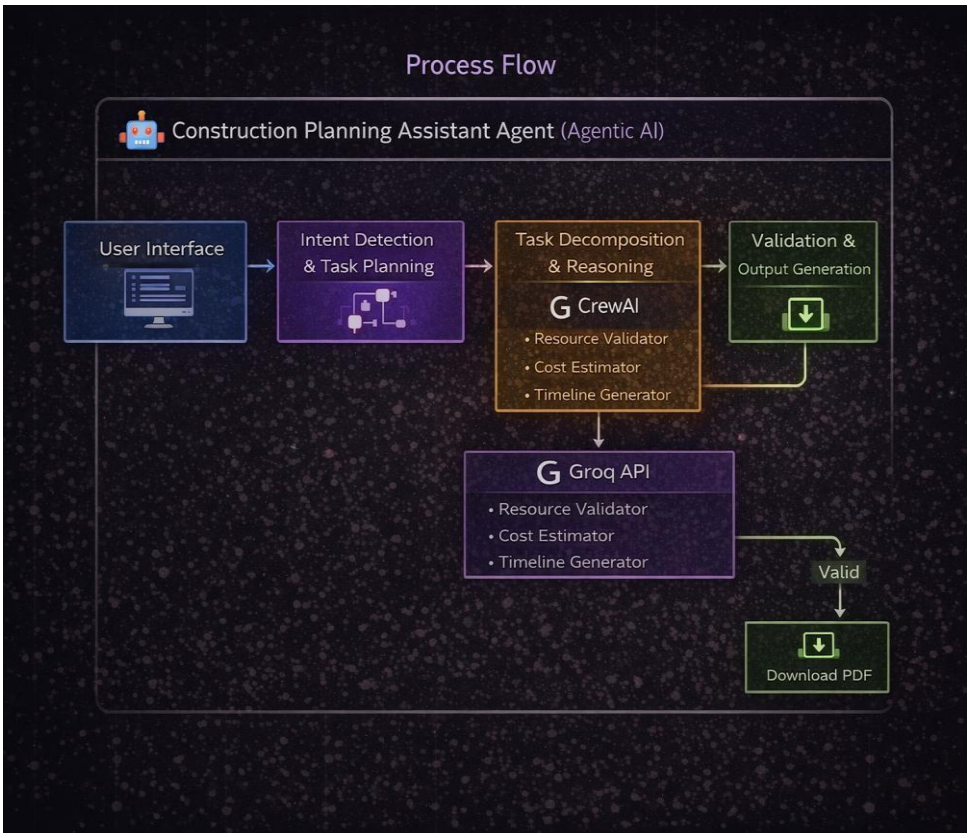
Based on your diagram:

- User enters a **construction goal**
- Orchestrator Agent performs: ○ Intent detection ○ Goal extraction
- System checks if query is construction-related
- Planner Agent activates: ○ Task decomposition ○ Multi-step reasoning
- Execution Loop: ○ Think → Act → Observe → Repeat
- Tools used:
  - ○ Permit Checker API (mock) ○ Resource Validator ○ Cost Estimator
    ○ Timeline Generator
- Context retrieved using: ○ Embeddings + FAISS (RAG) •
  Memory stores intermediate states
- Reporting Agent generates:
  - ○ Final plan + schedule

2.1Application Design



2.2 Process Flow



2.3 Information flow

- Input: Construction Goal

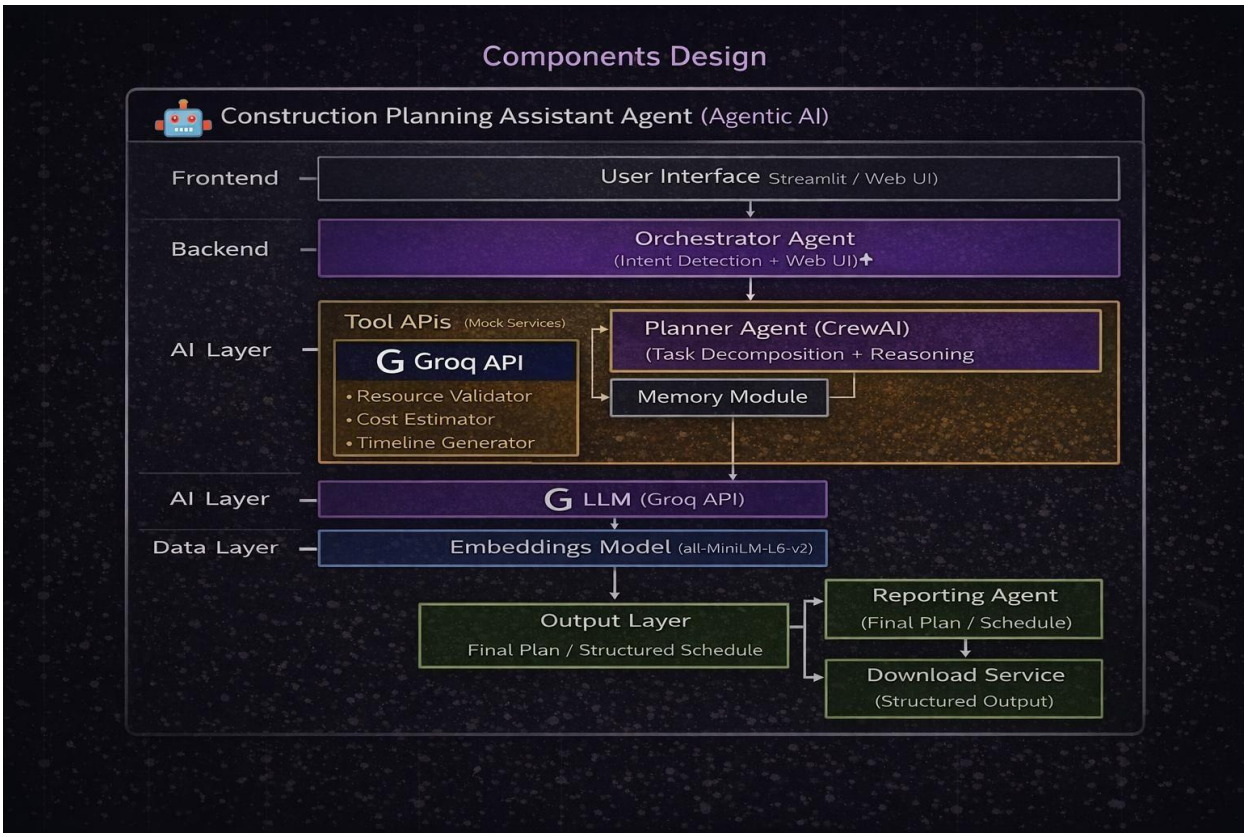
Process:

- Intent detection
- Task breakdown
- Tool interaction
- Context retrieval

Output:

- Execution Plan
- Timeline
- Resource Allocation

2.4 Components Design



2.5 API Catalogue.

1. LLM API

- Provider: Groq API (llama3-8b-8192)/ OpenAI
- Purpose: Reasoning + Planning

2. Embedding API

- Model: all-MiniLM-L6-v2
- Purpose: Vector creation

3. Tool APIs (Mock)

- Permit Checker API

- Resource Validator API
- Cost Estimator API
- Timeline Generator API

3. Data Design

3.1 Data Model

Input Data:

- goal (string)

Agent Data:

- tasks (list)
- steps (multi-step reasoning)

Vector Data:

- embeddings
- context chunks

Tool Data:

- permit_status
- resource_availability
- cost_estimate
- timeline

Output Data:

- final_plan
- schedule

3.2 Data Access Mechanism

Defines how data is retrieved, processed, and used.

- UI → Orchestrator
- Orchestrator → Planner Agent
- Planner → Tools + Vector DB
- Tools → Planner
- Planner → Reporting Agent
- Output → UI

4. Interfaces

This section describes the interfaces between different system components including the **User Interface, LLM services and supporting backend modules.**

- User Interface (Web App)
- Agent Communication Interface (CrewAI)
- Tool Integration Interface
-
- Vector DB Interface (FAISS)
- LLM Interface

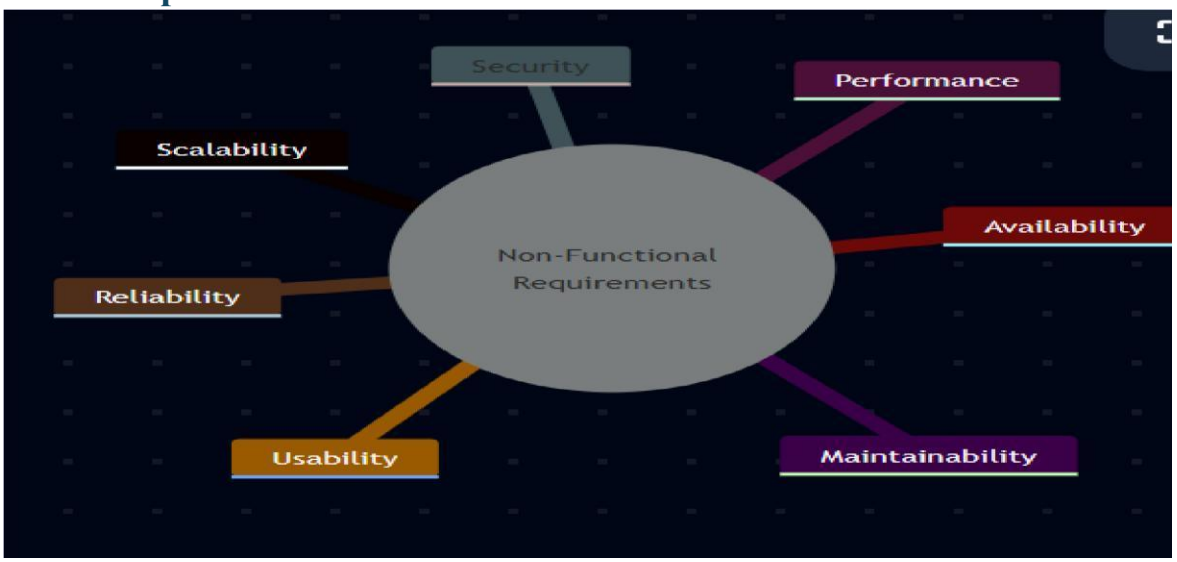
5 .Non-Functional Requirements

Non-functional requirements define the **quality attributes** of the system.

Key NFRs:

- Scalability → Supports multiple planning tasks
- Performance → Fast reasoning loops
- Security → API keys in .env
- Reliability → Stable agent execution
- Maintainability → Modular agent design

NFR Mindmap



5.1 Security Aspects

- API keys stored in .env file
- No hardcoding of credentials
- HTTPS-based API communication
- No persistent user data storage

5.2 Performance Aspects

Performance ensures fast response time and smooth user experience.

- Lightweight UI framework
- Prompt optimization

- Efficient API calls
- Minimal runtime memory usage

6. References

CrewAI/LangChain
FAISS – Facebook AI Similarity Search Documentation https://faiss.ai/ **Sentence Transformers – all-MiniLM-L6-v2 Model Documentation**
https://www.sbert.net/docs